

TP1 J1

Exercice 1 — Découverte de la Spark UI

Question 1 :

Dans Jobs, on voit les jobs exécutés par Spark et leur état.

Question 2 :

Les Stages sont les différentes étapes d'un Job.

Question 3 :

Les Task sont les petites tâches exécutées par Spark sur les données.

Question 4 :

Dans Environment, on voit la configuration de Spark : Java, Scala, propriétés Spark, Hadoop, etc.

Question 5 :

Dans Executors, on voit les ressources utilisées : CPU, mémoire, tâches, données, etc.

Question 6 :

- Job = travail complet
- Stage = étape du Job
- Task = tâche exécutée dans un Stage

Exercice 2 — Manipulation du DataFrame

```
] : df.show()
```

```
+-----+
|   name|age|
+-----+
|   Alice| 29|
|    Bob| 31|
|Catherine| 35|
|   David| 41|
|    Emma| 27|
|    Frank| 38|
+-----+
```

```
] : df.printSchema()
```

```
root
 |-- name: string (nullable = true)
 |-- age: long (nullable = true)
```

```
] : df.count()
```

```
] : 6
```

```
] : df.select("name").show()
```

```
+-----+
|   name|
+-----+
|   Alice|
|    Bob|
|Catherine|
|   David|
|    Emma|
|    Frank|
+-----+
```

```
)]: df.filter("age > 30").show()
```

```
+-----+----+
|      name|age|
+-----+----+
|      Bob| 31|
|Catherine| 35|
|      David| 41|
|      Frank| 38|
+-----+----+
```

Partie 3 :

Question : Combien de partitions votre DataFrame possède-t-il ?

Il y a qu'une seule partitions.

Exercice 3 :

Question 1 :

Pour répartir les données et permettre à Spark de travailler en parallèle.

Question 2 :

Cela permet d'utiliser plusieurs ressources en même temps et donc d'accélérer les traitements.

Question 3 :

Non, trop de partitions peut aussi ralentir le traitement à cause des coûts de gestion.

Question 4 :

Les ressources sont moins utilisées et le traitement peut être plus lent.

Question 5 :

Il y a plus de tâches à gérer, ce qui peut créer du temps supplémentaire et ralentir Spark.

Partie 4 :

Question :

Un nouveau Job est-il exécuté ?

Non aucun nouveau Job n'est exécuté.

Spark a-t-il déjà exécuté le traitement ?

Non, Spark n'a pas encore exécuté le traitement.

Exercice 4 :

Question 1 :

La Lazy Evaluation signifie que Spark n'exécute pas directement les transformations. Il attend une action pour lancer le calcul.

Question 2 :

Cela permet à Spark d'optimiser les calculs et d'éviter des traitements inutiles.

Question 3 :

Une transformation prépare un nouveau DataFrame, alors qu'une action lance réellement le calcul.

Question 4 :

`filter()` et `select()` .

Question 5 :

`show()` et `count()` .

Exercice 5 :

Question 1 :

Il y a 1 Job

Question 2 :

Il y a 1 stage.

Question 3 :

Il y a plusieurs task, selon le nombre de partitions.

Question 4 :

Un Job est composé de Stages, et les Stages sont composés de Tasks.

Question 5 :

Spark lance réellement le calcul et exécute les transformations nécessaires.

Question 6 :

Pour répartir le travail et permettre son exécution en parallèle.

Partie 6 :

Mission 1 :

```
[10]: df.show()
```

```
+-----+-----+-----+
|   city| product|amount|
+-----+-----+-----+
|   Paris|  Laptop| 1200|
|   Paris|   Mouse|   25|
|   Lyon|  Laptop| 1100|
|   Lyon|Keyboard|   80|
|Marseille| Laptop| 1300|
|Marseille|   Mouse|   30|
|   Paris|Keyboard|   75|
|   Lyon|   Mouse|   20|
+-----+-----+-----+
```

```
[11]: df.printSchema()
```

```
root
|-- city: string (nullable = true)
|-- product: string (nullable = true)
|-- amount: long (nullable = true)
```

Mission 2 :

```
[12]: df.filter(df.amount > 100).show()
```

```
+-----+-----+-----+
|   city|product|amount|
+-----+-----+-----+
|   Paris| Laptop| 1200|
|   Lyon| Laptop| 1100|
|Marseille| Laptop| 1300|
+-----+-----+-----+
```

Mission 3 :

```
df.groupBy("city").sum("amount").show()
```

city	sum(amount)
Marseille	1330
Paris	1300
Lyon	1200

Mission 4 :

```
df.groupBy("city").sum("amount").orderBy("sum(amount)").show()
```

city	sum(amount)
Lyon	1200
Paris	1300
Marseille	1330

Mission 5 :

```
df.rdd.getNumPartitions()
```

```
: 1
```

```
df = df.repartition(3)
```

```
df.rdd.getNumPartitions()
```

```
: 3
```

```
:
```

Mission 6 :

Jobs : 26

Stages : 1 stage par job

tasks : 73

Shuffle : 466 B

Executor : 1

Question final :

Que se passe-t-il dans Spark entre l'écriture de `df.filter(...)` et l'exécution de `df.show()` ?

Spark attend `df.show()` pour exécuter le `filter` et lancer le calcul.